# Sales Performance Dashboard

Final Project Report

**Mohamed Moin Peer**

MSc Data Analytics, Queen's University Belfast

Portfolio project | 2026

## Abstract

This project delivers a self-service sales dashboard for a synthetic UK retailer. A Python generator builds a SQLite database of 48,994 order lines across 2024-2025. A FastAPI backend exposes nine endpoints, all sharing one validated filter model, and a Vue 3 and Chart.js frontend presents KPI cards with period-on-period change, a revenue and profit trend, clickable regional and category breakdowns, and a top-products table with CSV export. Twenty-six automated tests check data integrity, reconcile every breakdown to the KPI totals and verify the API contract. The dashboard shows 2025 revenue of £4.07m (+9.5% on 2024), growth led by online sales (+22.1%), a strong Q4 peak (33% of annual revenue) and a low-margin Electronics category that brings 18.2% of revenue but only 8.6% of profit.


## 1. Introduction and aims

Sales managers repeatedly ask the same questions: how are we performing against the previous period, where is growth coming from, which regions and products matter most, and where is margin being lost. Answering them usually means an analyst running ad-hoc queries. The aim was to build a dashboard that answers these questions instantly and consistently.

Specific objectives were to:

- generate a realistic dataset with known business patterns;
- design a REST API whose endpoints always describe the same, validated slice of data;
- compare each period fairly with the previous comparable period;
- build an interactive, responsive frontend with filters and cross-filtering;
- prove the numbers are correct with automated reconciliation and API tests;
- turn the numbers into clear business recommendations.

## 2. Architecture

The system has four layers. generate_data.py creates the SQLite database. queries.py holds all SQL and the Filters object. api.py is the FastAPI application, which serves both the JSON endpoints and the static frontend. The frontend (index.html, app.js, styles.css) calls the API and draws the charts. A separate export_results.py writes summaries, charts and insights to outputs/. Keeping all SQL in one module means moving to a production database such as PostgreSQL would affect only that file.

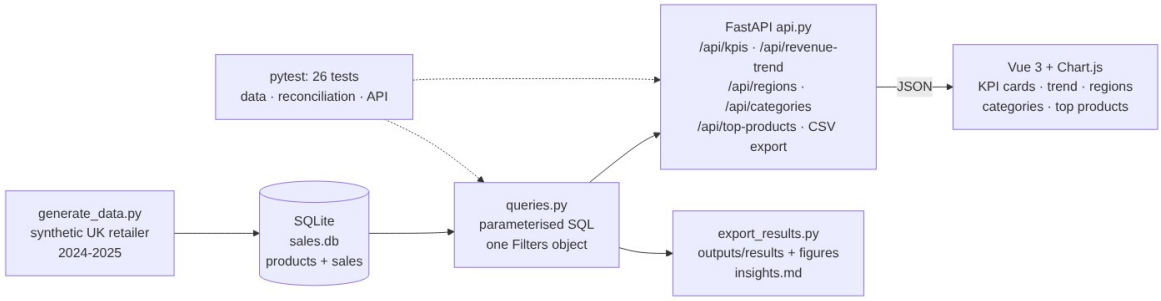



*Figure 1. System architecture.*

## 3. Data

Because real sales data is confidential, a synthetic dataset was generated with a fixed random seed, so every user obtains identical figures. It contains 40 products in five categories and 48,994 order lines over two years, across eight UK regions, two channels (online and in-store) and two customer segments.

Realistic patterns were built in deliberately: online revenue grows about 22% a year while in-store is roughly flat; November and December are peak months and January and February are quiet; in-store sales rise at weekends; discounts are more frequent in November and January; and Electronics carries a much higher cost ratio than other categories. Two simplifications should be noted: each order contains a single product line, so basket analysis is not possible, and no regional growth differences were designed, so any observed differences between regions are random variation.

| Table | Rows | Key columns |
|---|---|---|
| products | 40 | product_id, product_name, category, list_price, unit_cost |
| sales | 48,994 | order_id, order_date, region, channel, segment, product_id, quantity, unit_price, discount, revenue, cost, profit |

Business rules are enforced in the schema with CHECK constraints (positive quantity, discount between 0 and 1, valid channel and segment values), and indexes on date, region and product support fast filtered queries. Revenue is defined as quantity × unit price × (1 − discount), and profit as revenue minus cost.

## 4. Backend design

### 4.1 One filter model

Every dashboard endpoint accepts the same optional filters: start date, end date, region, channel and category. A shared FastAPI dependency validates them once. Incorrect types (for example an invalid date) return HTTP 422; a start date after the end date, or a region, channel or category that does not exist, returns HTTP 400 with an explanatory message. The validated values become a single immutable Filters object passed to every query, guaranteeing that KPI cards, trend, breakdowns and tables always describe exactly the same data.

```
def get_filters(start: date | None = None, end: date | None = None,
                region: str | None = None, ...) -> Filters:
    if start > end: raise HTTPException(400, 'start must be on or before end')
    if region and region not in OPTIONS['regions']: raise HTTPException(400, ...)
    return Filters(start, end, region, channel, category)

@app.get('/api/kpis', response_model=KPIs)
def kpis(f: Filters = Depends(get_filters)): return queries.kpis(con, f)
```

## 4.2 Endpoints

| Endpoint | Extra parameters | Returns |
|---|---|---|
| /api/filters | - | Allowed regions, channels, categories and date range |
| /api/kpis | - | Revenue, profit, margin, orders, units, AOV and % change |
| /api/revenue-trend | grain: day/week/month/quarter | Revenue, profit and orders per period |
| /api/regions | - | Revenue, profit, orders, margin % and share % per region |
| /api/categories | - | As above per category, with units |
| /api/channels | - | Online vs in-store |
| /api/top-products | limit 1-40; sort: revenue/profit/units | Top products with margin % |
| /api/export/products.csv | - | CSV of all products for the current filters |
| /api/health | - | Status and number of sales lines |

Pydantic response models validate outgoing data and allow FastAPI to generate interactive documentation at /docs. CORS is enabled for GET requests so the frontend can also be served from a separate development server.

## 4.3 Previous-period comparison

A percentage change is only meaningful against a fair baseline. When the selection consists of whole calendar months, it is compared with the same number of whole months immediately before (2025 with 2024; Q4 with Q3). Otherwise it is compared with the same number of days immediately before. Where the earlier period has no data, the change is returned as null and the card displays "no comparison period" rather than a misleading figure.

The first implementation always went back the same number of days. Because 2024 was a leap year, 2025 was compared with 2 January to 31 December 2024. The calendar-month rule fixed this, and a unit test now covers whole years, quarters and custom ranges.

## 4.4 Safety and correctness

User-supplied values are only ever passed to SQLite as parameters, never formatted into SQL strings, and the sort column for top products is selected from a fixed whitelist. A test passes the string "London' OR '1'='1" as a region directly to the query layer and confirms that it matches no rows. Empty selections return zeros rather than errors, so the interface degrades gracefully.

## 5. Frontend design

The frontend uses Vue 3 without a build step: three files plus Vue and Chart.js bundled in frontend/vendor/, so it works offline and requires no Node.js installation. A package.json documents the exact library versions and a script can refresh the bundled copies.

- Reactive filters: a watcher on the query string reloads all panels in parallel.
- Stale-response protection: each load receives a request ID, and responses from older requests are ignored.
- Chart objects are kept outside Vue's reactivity system and updated in place for performance.
- Date presets (All time, each year, Q4, last 90 days) alongside free date selection.
- KPI cards show change against the previous period, coloured green or red, with the comparison period in a tooltip.
- Cross-filtering: clicking a region or category bar filters the whole dashboard. The clicked chart ignores its own filter so that all bars remain visible with the selection highlighted.
- Top-products table sortable by revenue, profit or units, with low margins (below 30%) highlighted, and CSV export.
- Responsive layout down to phone width, and clear error messages if the API is unavailable.

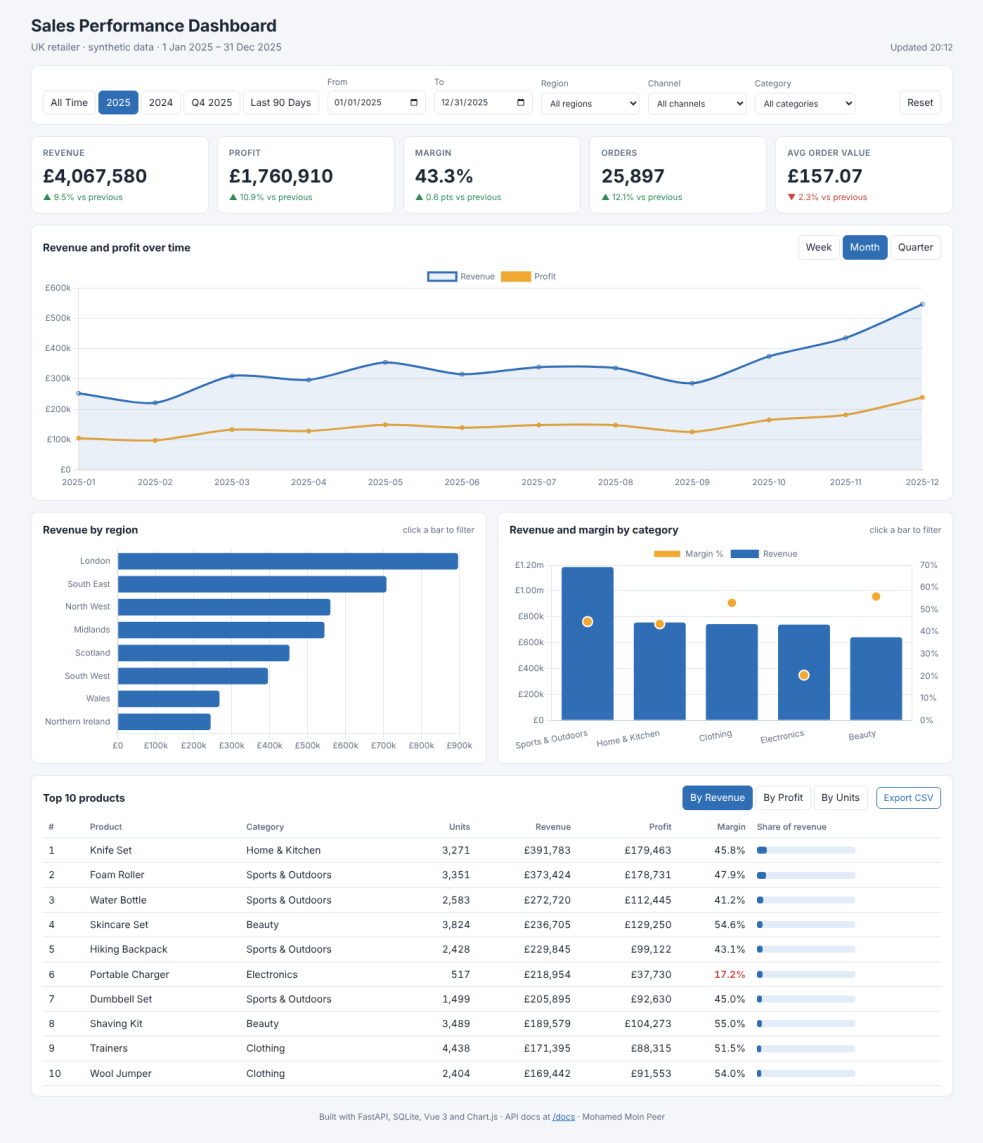



*Figure 2. The dashboard showing 2025.*

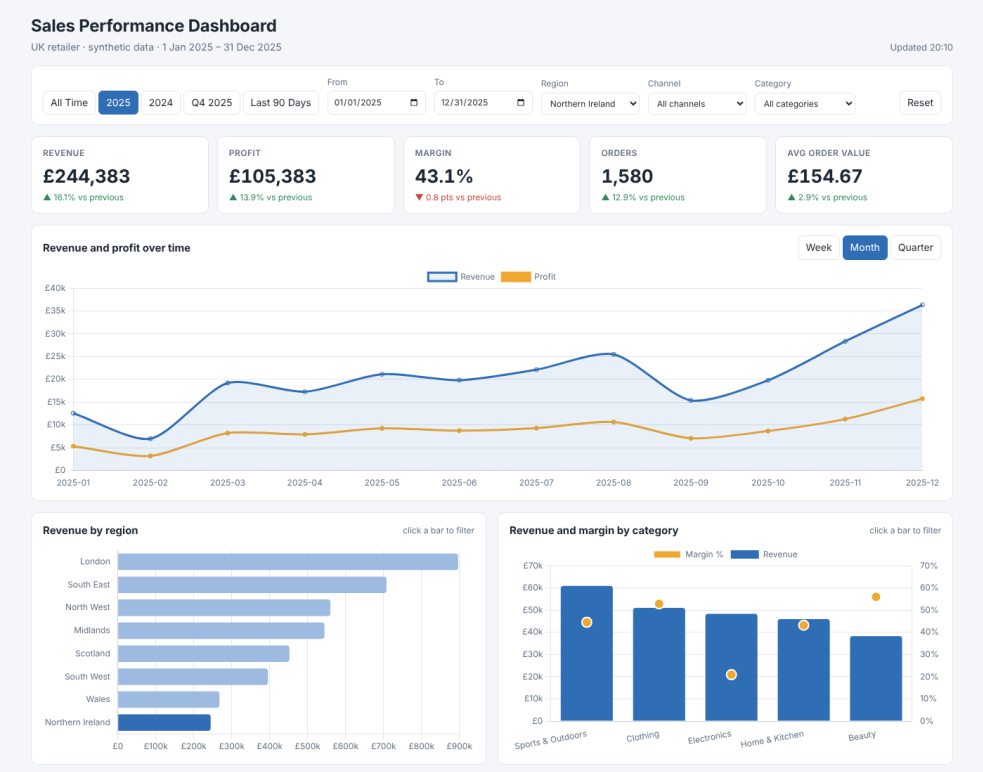



*Figure 3. Cross-filtering: 2025 filtered to Northern Ireland.*

# 6. Findings

## 6.1 Headline performance

| Metric | 2024 | 2025 | Change |
|---|---|---|---|
| Revenue | £3.71m | £4.07m | +9.5% |
| Profit | £1.59m | £1.76m | +10.9% |
| Margin | 42.7% | 43.3% | +0.6 pts |
| Orders | 23,097 | 25,897 | +12.1% |
| Average order value | £160.81 | £157.07 | -2.3% |

Revenue and profit both grew, with margin slightly improved. Orders grew faster than revenue while average order value fell, so growth came from more orders rather than larger baskets.

## 6.2 Channels

Online revenue grew 22.1% while in-store changed -1.5%. Online overtook in-store during 2025 and now accounts for 52% of revenue.

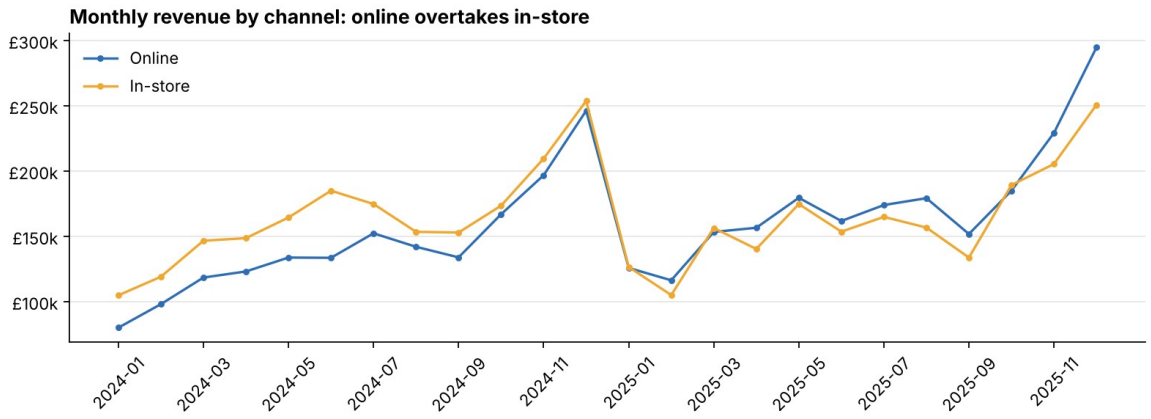



*Figure 4. Monthly revenue by channel.*

## 6.3 Seasonality

Revenue rises from October to a December peak and falls sharply in January and February. Q4 2025 delivered 33% of annual revenue in a quarter of the year.

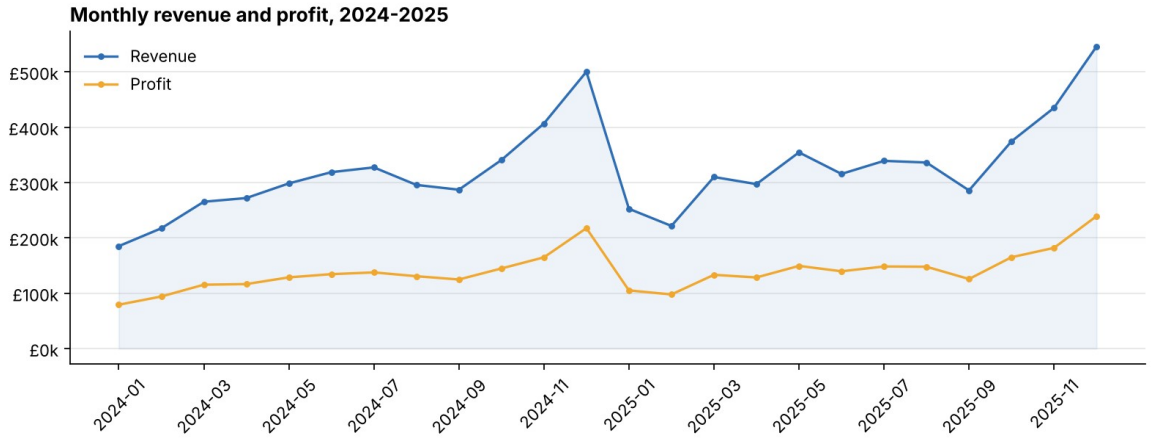



*Figure 5. Monthly revenue and profit, 2024-2025.*

## 6.4 Category margins

| Category | Revenue | Share of revenue | Share of profit | Margin |
|---|---|---|---|---|
| Sports & Outdoors | £1.19m | 29.1% | 30.0% | 44.5% |
| Home & Kitchen | £757k | 18.6% | 18.7% | 43.5% |
| Clothing | £744k | 18.3% | 22.4% | 53.0% |
| Electronics | £740k | 18.2% | 8.6% | 20.4% |
| Beauty | £642k | 15.8% | 20.4% | 55.8% |

Electronics generates 18.2% of revenue but only 8.6% of profit, with a margin of 20.4% against 44-56% for the other categories. Beauty and Clothing contribute a larger share of profit than of revenue. Portable Charger is a top-ten product by revenue at a margin of about 17%.

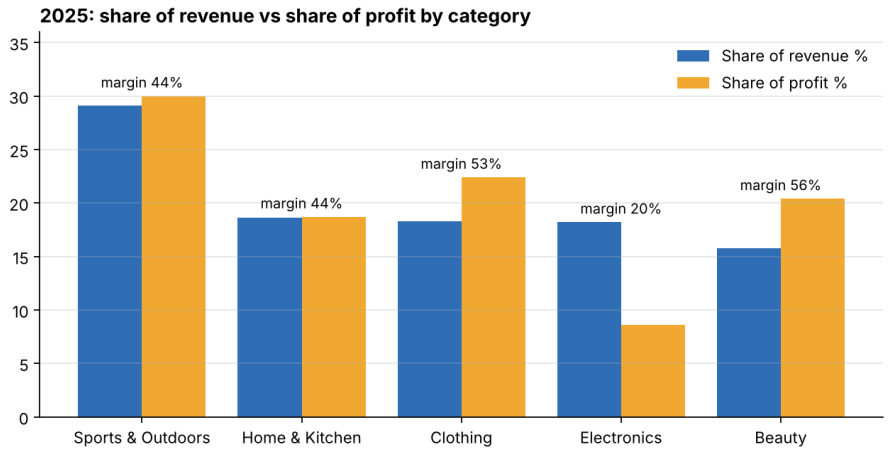



*Figure 6. Share of revenue versus share of profit by category.*

## 6.5 Regions

London is the largest region (22.0% of revenue), followed by the South East (17.4%). All regions grew, by between 6.8% and 16.1%. Because no regional differences were designed into the data, these variations are random. On real data, regional differences should be compared with normal month-to-month variation before any action is taken.

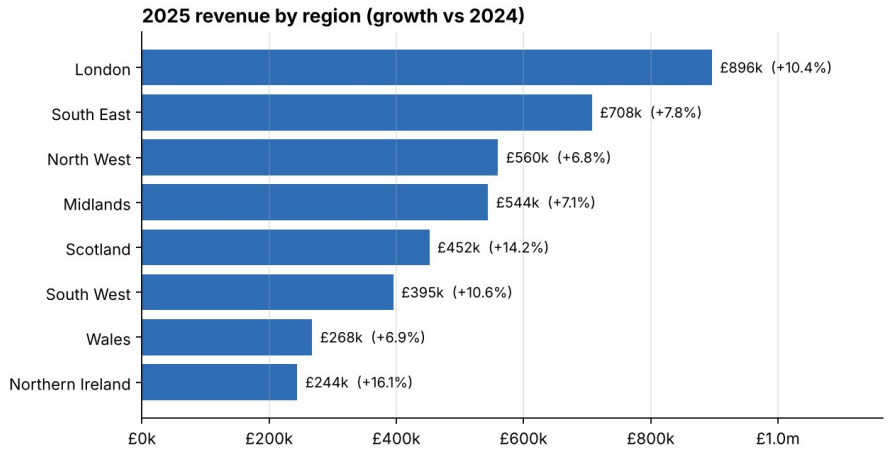



*Figure 7. 2025 revenue by region with growth on 2024.*

# 7. Testing

Twenty-six pytest tests run against a fresh database created in a temporary location, never the working database.

| Area | Tests | What is checked |
|---|---|---|
| Data | 6 | Row counts; revenue and profit formulas on every line; no orphan products; full |

| | | |
|---|---|---|
| | | date coverage; reproducibility; December revenue exceeds 1.5 × January |
| Queries | 10 | Regions, categories, channels and trend reconcile to KPI revenue for three filter combinations; KPIs match independent SQL; previous-period rule; null change without prior data; shares sum to 100%; sorting and limits; empty selections; SQL injection |
| API | 10 | Health; filter options; filtered KPIs; 400 for unknown values and reversed dates; 422 for invalid types; trend grains; limit validation; CSV export; frontend served |

The reconciliation tests are particularly valuable: if a join were ever to duplicate rows or a filter were applied inconsistently, the breakdowns would no longer add up to the KPI totals and the tests would fail.

## 8. Recommendations

- Prioritise online: allocate marketing spend and stock toward the online channel, and develop stores for click-and-collect and returns.
- Plan for Q4: secure stock and seasonal staff early; treat January promotions as a margin cost.
- Protect margin: avoid discounting Electronics and favour higher-margin Beauty and Clothing in promotions; review pricing and supplier costs for low-margin best-sellers such as Portable Charger.
- Grow basket size: test bundles and add-on recommendations, since average order value fell.
- Treat regional differences cautiously until they are shown to exceed normal variation.

## 9. Limitations and future work

- Synthetic data with single-line orders; real data would add noise and allow basket analysis.
- No authentication or role-based access.
- Queries run directly on SQLite without caching, which is adequate for about 49,000 rows but not for millions.
- Future work: connect PostgreSQL or a data warehouse, add caching, add a forecast to the trend chart, add login with region-level access, and deploy with continuous integration (GitHub Actions).

## 10. Conclusion

The project delivers a complete, tested, full-stack analytics application. A single validated filter model, a carefully defined comparison period and reconciliation tests make the numbers consistent and trustworthy. The interactive frontend turns them into answers a manager can act on: online is driving growth, Q4 dominates the year, and Electronics discounting is the most expensive place to lose margin.

## Appendix: how to run

```
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
```

```
pip install -e .
python -m sales_dashboard all
pytest
python -m sales_dashboard serve   ->   http://127.0.0.1:8000
```